

Für dieses Projekt steht Ihnen in StudIP folgender Programmcode zur Verfügung:  
uap25-pro02-python.zip

## Aufgabe 1 : Grammatik der TRIPLA (4 Punkt(e))

Betrachten Sie die bisherige Grammatik der in der Vorlesung vorgestellten Programmiersprache: **TR**ier **P**rogramming **L**anguage (TRIPLA).

```
1  E -> let D in E
2      | ID
3      | ID ( A )
4      | E AOP E
5      | ( E )
6      | CONST
7      | ID = E
8      | E ; E
9      | if B then E else E
10     | while B do { E }
11  A -> E | A , E
12  D -> ID ( V ) { E }
13     | D D
14  V -> ID | V , V
15  B -> E | E RELOP E
```

```
16  ID :   Bezeichner = [A-Za-z_][A-Za-z0-9_]*
17  CONST: Positive, ganze Zahl = 0 | [1-9][0-9]*
18  AOP:   Operatoren (+, -, *, /)
19  RELOP: Vergleichsoperatoren (==, !=, <, >, <=, >=)
```

1. Erweitern Sie die Grammatik um die Terminale `true` und `false`.
2. Erweitern Sie die Grammatik um eine weitere Menge an Operatoren LOP (logical operations). Sie stellt die logischen Operatoren `||` (OR), `==` (EQ), `!=` (NEQ) sowie `&&` (AND) dar.
3. Erweitern bzw. ändern Sie die Grammatik so, dass nur noch logische Ausdrücke als Bedingung von `if` und `while` erlaubt sind und diese auch in runde Klammern gefasst werden können. Zu den logischen Ausdrücken zählen insbesondere `true` und `false`. Zahlenwerte sollen nicht mehr als Bedingung erlaubt sein. Hinweis: Sie müssen die Produktionen des Nichtterminals B anpassen.

Demnach sind folgende Programme gültige Wörter der erzeugten Sprache:

```
1 if ( true ) then 1 else 0 // Erwartetes Ergebnis: 1
2
3 if ( true && false ) then 1 else 0 // Erwartetes Ergebnis: 0
4
5 if ( true || false ) then 1 else 0 // Erwartetes Ergebnis: 1
6
7 if ( 1 > 2 ) then 1 else 0 // Erwartetes Ergebnis: 0
8
9 if ( 2 > 3 + 5 ) then 1 else 0 // Erwartetes Ergebnis: 0
10
11 if ( 1 > 2 || 3 < 5 ) then 1 else 0 // Erwartetes Ergebnis: 1
12
13 if ( 2 == 0 == false ) then 1 else 0 // Erwartetes Ergebnis: 1
14
15 let func(a,b) {
16     while ( a > 0 && b != a ) do {
17         b = b + 1;
18         a = a - 1
19     }
20 }
21 in func(10, 8) // Erwartetes Ergebnis: 9
```

Mit „Erwartetes Ergebnis“ ist die gedankliche Ausführung des Codes gemeint. **Sie sollen ausdrücklich die Auswertung des Codes nicht implementieren!**

Keine Worte der Sprache sind beispielsweise:

```
1 // Parser sollte Fehler ausgeben
2 if ( true + 9 ) then 1 else 0
3
4 // Parser sollte Fehler ausgeben
5 if ( 0 ) then 1 else 0
6
7 // Parser sollte Fehler ausgeben
8 if ( 2 > 3 && 7 ) then 1 else 0
9
10 // Parser sollte Fehler ausgeben
11 let f(a, b) {
12     if (a = 3) then
13         1
14     else
15         0
16 } in 3
```

## Aufgabe 2 : Implementierung Lexer und Parser (12 Punkt(e))

Implementieren Sie einen Lexer und Parser für die angepasste Grammatik mit Hilfe des Python-Pakets ply ( <https://www.dabeaz.com/ply/ply.html>).

Um Ihnen diese Aufgabe etwas zu erleichtern, bieten wir Ihnen eine rudimentäre Implementierung eines Parsers mit ply zum Herunterladen auf StudIP an, die Ihnen die Funktionsweise von ply veranschaulicht.

## Aufgabe 3 : Abstrakter Syntaxbaum (4 Punkt(e))

Erweitern Sie ihren Parser so, dass er eine interne Datenstruktur (Parse-Baum, Abstrakter Syntaxbaum) aufbaut. Schreiben Sie zusätzlich eine Routine, die den Baum im DOT-Format in eine Datei exportiert.

### Hinweise

- Da die Grammatik der TRIPLA zunächst keine Operator-Präzedenz vorsieht, können Sie diese mit Hilfe des Schlüsselwortes `precedence` für die jeweiligen Operatoren in der Datei `triplayacc.py` nachrüsten.
- In der TRIPLA kann man Parameter als lokale Variablen beispielweise wie folgt ausnutzen:

```
1 let func(a, b) {  
2     a = b + 1  
3 }  
4 in func(10, 8)
```

- Wiki Artikel zum DOT Format:  
[https://en.wikipedia.org/wiki/DOT\\_\(graph\\_description\\_language\)](https://en.wikipedia.org/wiki/DOT_(graph_description_language))
- Zur Anzeige der Graphen können Sie Werkzeuge wie Graphviz (<https://www.graphviz.org/>) oder Gephi (<https://gephi.org/>), sowie den Online-Dienst <https://dreampuf.github.io/GraphvizOnline> verwenden.
- **Beachten Sie, dass die Auswertung des Programms Teil der dritten Projektaufgabe ist. In der zweiten Projektaufgabe soll lediglich ein abstrakter Syntaxbaum aufgebaut werden.**

**Abgabetermin:** Donnerstag, der 27.11.2025 zum Vorlesungsbeginn via StudIP

**Abgabeformat:** Die Abgabe der Übungsblätter erfolgt über Stud.IP. Bitte beachten Sie die folgenden Hinweise. Abgaben, die nicht dem beschriebenen Format entsprechen, werden ignoriert und mit 0 Punkten bewertet:

- Bei **normalen Übungsblättern**:

Die Abgabe erfolgt als **einzelne PDF-Datei** (keine Ordner, kein ZIP-Archiv, nicht in mehreren Teilen, keine anderen Dateiformate). **Bitte vermerken Sie in dieser PDF-Datei gut lesbar Ihren Namen und Ihre Matrikelnummer.** Verwenden Sie folgendes Schema für den Dateinamen der PDF-Datei: `uap25-projxx-123456` (xx durch Nummer des entsprechenden Übungsblattes ersetzen, z.B. `ueb01` für das erste Blatt, und `123456` durch Ihre Matrikelnummer ersetzen).

- Bei **Übungsblättern mit Programmieraufgaben**:

Neben der evtl. vorhandenen PDF-Datei soll nur der Quellcode abgegeben werden (also keine `class`-Dateien o.Ä.). **Bitte vermerken Sie in jeder Quellcodedatei Ihren Namen und Ihre Matrikelnummer als Kommentar.** Packen Sie in diesem Fall alle benötigten Dateien (Quellcode und ggf. PDF-Datei) als **ZIP-Datei** mit dem Namen `uap25-projxx-123456.zip` (xx durch Nummer des entsprechenden Übungsblattes ersetzen, z.B. `ueb01` für das erste Blatt, und `123456` durch Ihre Matrikelnummer ersetzen).